torchtest2: testing PyTorch models before you train them

A practical tutorial for students and PhD students

Grzegorz Gwardys

Warsaw University of Technology ▪ Promity

Why test a model before training?

Silent bugs

Many deep-learning bugs do not crash. The code runs, the loss goes down a bit, and the bug shows up—if ever—after days of GPU time.

- a layer that is defined but never used in `forward`
- an optimizer created before the head was replaced
- `.detach()` / `.data` / `.item()` that cuts the graph
- an RNN or attention layer that treats the batch as time (`batch_first` mix-up)
- an attention mask that is ignored
- dropout that stays on at inference

None of these raises an exception. Most of them do not even stop the loss from decreasing.

Folklore, packaged into one call

Experienced practitioners run sanity checks by hand:

- “check the loss at initialisation”, “overfit a single batch” (Karpathy)
- “check that all variables change” (mltest, torchtest)
- “check that samples do not influence each other” (Krokotsch)

torchtest2 runs 18 such checks on one batch, in seconds, with one call, and explains every finding:

```
report = torchtest2.check_model(model, inputs, targets,
                                loss_fn=F.cross_entropy, optimizer=optimizer)
print(report)   # PASS / WARN / FAIL / SKIP for each check, with a message
```

It is meant to live in your unit tests, next to the model code—before a training pipeline even exists.

First steps

Installation

```
pip install torchtest2
```

- Python $\geq$ 3.10, PyTorch $\geq$ 2.1, no other dependencies
- works on CPU and GPU; inputs and targets are moved to the model's device

The minimum call needs only a model and a batch:

```
torchtest2.check_model(model, inputs)
```

Without a loss function the loss-based checks are skipped and the gradient checks use a random projection of the outputs instead. Give the loss and the optimizer you really train with to get all checks.

A model with a bug

```
import torch, torch.nn as nn, torch.nn.functional as F
import torchtest2

class Classifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(nn.Linear(32, 64), nn.ReLU())
        self.norm = nn.LayerNorm(64)          # defined...
        self.head = nn.Linear(64, 10)

    def forward(self, x):
        return self.head(self.encoder(x))     # ...but never used
torch.manual_seed(0)
model = Classifier()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
x, y = torch.randn(16, 32), torch.randint(0, 10, (16,))

report = torchtest2.check_model(model, x, y, loss_fn=F.cross_entropy, optimizer=optimizer)
print(report)
```

The report

```
torchtest2 report for Classifier  (parameters: 2,890, trainable: 2,890, device: cpu, batch size: 16, objective: loss_fn)
  PASS  forward             output: Tensor[16, 10] float32
  SKIP  output_shape        no output_shape given
  PASS  output_finite       no NaN/inf in the outputs
  SKIP  output_range        no output_range given
  PASS  output_varies       outputs differ between samples
  PASS  eval_determinism    eval-mode forward passes are reproducible
  PASS  loss                loss = 2.323 (loss_fn)
  PASS  initial_loss        initial loss 2.323 is close to ln(10) = 2.303
  PASS  optimizer_coverage  the optimizer covers every trainable parameter
  FAIL  gradient_flow       2/6 trainable parameter(s) get no gradient from the loss_fn (unused in
                            forward, or the graph is cut by .detach()/.item()/no_grad): norm.weight, norm.bias
  PASS  gradient_magnitude  total gradient norm 0.949
  FAIL  params_change       2/6 trainable parameter(s) did not change after an optimizer step: norm.weight, norm.bias
  SKIP  frozen_params       no frozen parameters (requires_grad=False or frozen=[...])
  PASS  input_dependency    all 1 input tensor(s) influence the output
  PASS  batch_independence  outputs of a sample do not depend on other samples in the batch (eval mode)
  PASS  overfit_batch       loss 2.323 -> 0.2244 in 85 step(s)
  SKIP  gpu                 require_gpu=False
  11 passed, 0 warnings, 2 failed, 4 skipped
```

Every finding names the offending parameters and suggests likely causes.

Four statuses

Status	Meaning
PASS	nothing suspicious
WARN	suspicious, but can be legitimate: read the message
FAIL	almost certainly a bug
SKIP	not applicable (e.g. no loss given) or disabled

- `report.ok` is `True` when nothing FAILed (warnings are allowed)
- `strict=True` turns warnings into failures
- why warnings? Some findings are legitimate by design: a zero-initialised head gives zero gradients at init, a contrastive model compares samples on purpose

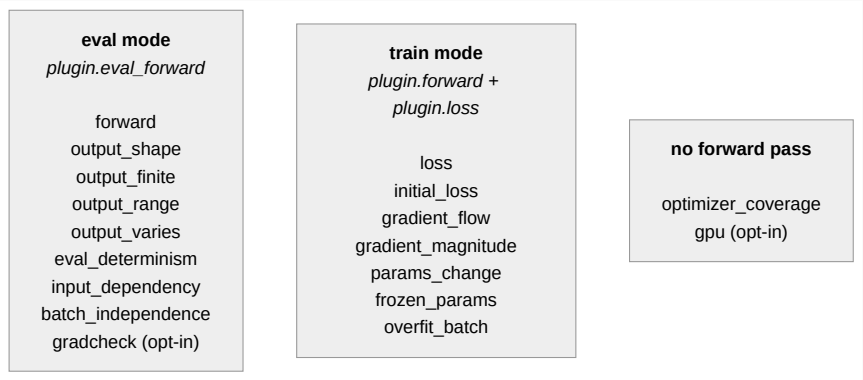
The most important checks

All 18 checks at a glance

Group	Checks	Catches
outputs	`forward`, `output_shape`, `output_finite`, `output_range`	crashes, wrong head size, NaN, missing sigmoid
	`output_varies`, `eval_determinism`	collapsed outputs, randomness at inference
loss	`loss`, `initial_loss`	unreduced loss, over-confident init
optimisation	`optimizer_coverage`, `params_change`, `frozen_params`	stale optimizer, `lr=0`, a "frozen" layer that trains
gradients	`gradient_flow`, `gradient_magnitude`	unused layers, cut graph, NaN
data flow	`input_dependency`, `batch_independence`	ignored inputs/masks, samples leaking
training	`overfit_batch`	anything that prevents learning
opt-in	`gradcheck`, `gpu`	wrong custom `backward`, training on CPU

The next slides show the checks that catch the most common silent bugs.

Which checks run the model, and how



Inference checks run in eval mode (so BatchNorm's batch statistics and dropout are off), training checks in train mode. The plugin section shows which plugin method each mode uses.

`optimizer_coverage`: the fine-tuning classic

```
model = nn.Sequential(nn.Linear(32, 64), nn.ReLU(), nn.Linear(64, 1000))  # "pretrained"
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
model[2] = nn.Linear(64, 10)              # new head for 10 classes - AFTER the optimizer

report = check_model(model, x, y, loss_fn=F.cross_entropy, optimizer=optimizer)
```

```
FAIL  optimizer_coverage  2 trainable parameter(s) are not in the optimizer: 2.weight, 2.bias. The optimizer
                          holds 2 tensor(s) that are not (or no longer) parameters of this model - were layers
                          replaced after creating it, or was it created for another model instance?
FAIL  params_change       2/4 trainable parameter(s) did not change after an optimizer step: 2.weight, 2.bias
```

- the new head is never trained; the loss still goes down a little
- `optimizer_coverage` compares parameter *identities* with the optimizer's parameter groups—it needs your optimizer

gradient_flow and params_change: a cut graph

```python
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = nn.Linear(32, 64)
        self.head = nn.Linear(64, 10)

    def forward(self, x):
        features = self.backbone(x).data     # legacy .data silently detaches
        return self.head(F.relu(features))
```

```
FAIL  gradient_flow  2/4 trainable parameter(s) get no gradient from the loss_fn (unused in forward, or the
                     graph is cut by .detach()/.item()/no_grad): backbone.weight, backbone.bias
```

- None gradient ⇒ FAIL (unused, or the graph is cut)
- all-zero gradient ⇒ WARN (zero-initialised layers are legitimate)
- params_change: one step with your optimizer must change every trainable parameter (catches lr=0, too)

`batch_independence`: samples must not talk to each other

```python
class Encoder(nn.Module):
    def __init__(self, dim=16, heads=4):
        super().__init__()
        self.attn = nn.MultiheadAttention(dim, heads)   # batch_first=False by default!
        self.out = nn.Linear(dim, dim)

    def forward(self, x):                                # x: (batch, tokens, dim)
        h, _ = self.attn(x, x, x)
        return self.out(h)

report = check_model(Encoder(), torch.randn(4, 6, 16))
```

```
FAIL  batch_independence  changing sample 0 changed the outputs of 3/3 other sample(s), e.g. [1, 2, 3]: the
                          model mixes information across the batch (view/reshape instead of transpose/permute,
                          a batch_first mix-up, softmax or normalisation over the batch dimension?)
```

How: replace sample 0 by sample 1 and check, in eval mode, that the outputs of samples $1 \dots N{-}1$ do not change. Works for token ids, masks and any output structure.

`input_dependency`: is every input actually used?

```python
class TwoTower(nn.Module):
    def __init__(self):
        super().__init__()
        self.user = nn.Linear(16, 8)
        self.item = nn.Linear(16, 8)

    def forward(self, x):                       # x = [user features | item features]
        u, i = x[:, :16], x[:, 16:]
        return (self.user(u) * self.item(u)).sum(-1)    # BUG: item tower gets user features
```

```
FAIL  input_dependency  16/32 features of inputs (per-sample shape (32,)) have no influence on the output,
                        e.g. at [(16,), (17,), (18,)] (wrong slice/split, a variable reused by mistake?)
```

- float inputs: gradient per feature; integer and boolean inputs (token ids, attention masks) are rolled along the batch and must change the output
- catches ignored attention masks in Hugging Face-style `dict` inputs too

Three more checks worth knowing

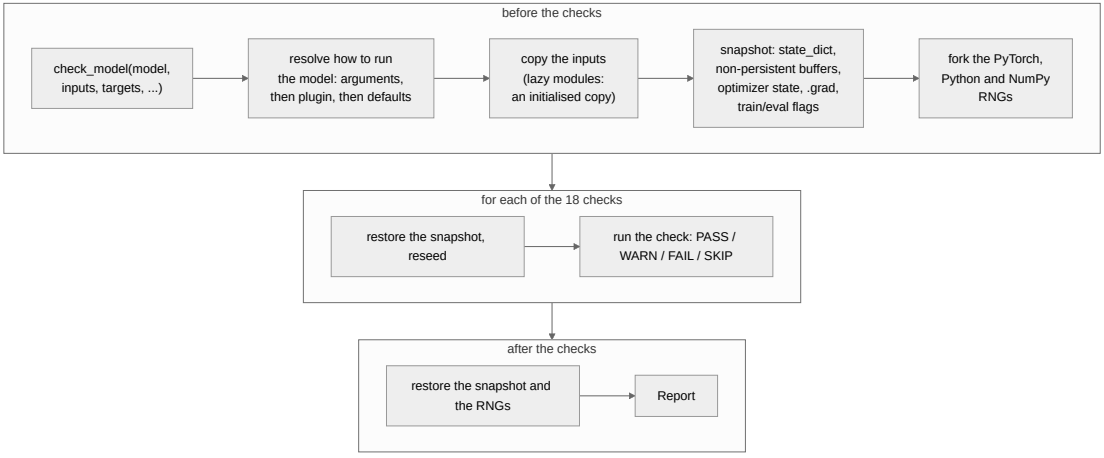
`eval_determinism` two forward passes in eval mode must agree. WARN for `F.dropout(x)` without `training=self.training`, or `torch.rand` in `forward`.

`initial_loss` cross-entropy at initialisation should be close to $\ln C$. WARN for an over-confident initialisation (or label leakage).

`overfit_batch` trains on the one batch until the loss falls to 10 % of its initial value; checks that gradients stay finite at every step. Catches anything that prevents learning, and NaN gradients that appear only after a few updates.

`overfit_batch` is the slowest check (up to `overfit_steps=200`); lower it, or `skip=["overfit_batch"]` while iterating.

What happens inside `check_model`



It does not touch your objects

torchtest2 restores the PyTorch state it touches after every check:

- parameters, buffers (also non-persistent ones), optimizer state, `.grad` tensors, train/eval flags, uninitialised lazy modules
- the PyTorch, Python and NumPy random generators
- your input tensors are copied, so in-place operations in `forward` cannot change them

So you can call it anywhere in a test suite—even on the model you are about to train.

Not restored: plain Python attributes that your own `forward` changes (e.g. a call counter).

Running checks in a test suite

pytest: one line per architecture

```python
import pytest, torch, torch.nn.functional as F            # tests/test_models.py
import torchtest2
from my_project.models import build_model            # your code

@pytest.mark.parametrize("arch", ["resnet18", "vit_tiny", "my_new_idea"])
def test_architecture_is_trainable(arch):
    torch.manual_seed(0)
    model = build_model(arch, num_classes=10)
    x, y = torch.randn(8, 3, 64, 64), torch.randint(0, 10, (8,))
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
    torchtest2.assert_model_ok(model, x, y, loss_fn=F.cross_entropy, optimizer=optimizer,
                               output_shape=(None, 10), overfit_steps=100)
```

```
pytest tests/test_models.py
```

`assert_model_ok` raises `ModelCheckError` (an `AssertionError`) listing the failed checks and the whole report, so pytest shows it as a test failure.

Tuning the checks to your model

```
# warnings are failures, too
torchtest2.assert_model_ok(model, x, y, loss_fn=F.cross_entropy, strict=True)

# a contrastive model compares the samples of a batch on purpose
torchtest2.assert_model_ok(model, x, skip=["batch_independence"])

# a frozen backbone must not change; outputs must be probabilities
torchtest2.assert_model_ok(model, x, y, loss_fn=F.nll_loss,
                           frozen=["backbone"], output_range=(0, 1))

# sequence-first RNN: inputs (time, batch, features), outputs (batch, classes)
torchtest2.assert_model_ok(model, x, batch_dim=1, output_batch_dim=0)
# assert on single checks
report = torchtest2.check_model(model, x, y, loss_fn=F.cross_entropy)
assert report["gradient_flow"].status is torchtest2.Status.PASS
```

Tip: use a small configuration of your architecture (few layers, narrow width) and a batch of 4–16 distinct samples. Most bugs do not depend on size.

The class-based API: `ModelTester` and `TestSuite`

```python
from torchtest2 import ModelTester, TestSuite

batch = [torch.randn(32, 1, 28, 28), torch.randint(0, 10, (32,))]
model = nn.Sequential(nn.Flatten(), nn.Linear(784, 64), nn.ReLU(), nn.Linear(64, 10))
optimizer = torch.optim.Adam(model.parameters(), lr=1e-2)

tester = ModelTester(model, F.cross_entropy, optimizer, batch)
tester.test_parameters_change()      # one test; raises ParametersChangeException
tester.test_batch_independence()     # raises BatchIndependencyException
report = tester.check()              # the full check_model report
suite = TestSuite(model, F.cross_entropy, optimizer, batch,
                  test_suite_config={"test_parameters_change": True,
                                     "test_input_dependency": True,
                                     "test_overfit": True})
results = suite.test_suite()         # runs ALL enabled tests, prints a summary -> {test: passed}
```

`ModelTester.test_suite()` stops at the first failure; `TestSuite` collects all results—useful in notebooks and reports.

Plugins

Why plugins?

Not every model is trained through `loss_fn(model(inputs), targets)`:

- detection models take the targets in training mode and return a dict of losses
- Hugging Face models compute their own loss from `labels=`
- Lightning modules define `training_step` and `configure_optimizers`
- your lab's multi-task models do something else again

A plugin answers "how is this family of models run?"—training forward, eval forward, loss, optimizer, batch dimensions.

The checks themselves stay the same for every model: a plugin never decides what is checked, only how the model is executed. That keeps reports comparable across model families.

Built-in plugins

Name	For	What it does
`"torchvision_detection"`	Faster/Mask R-CNN, RetinaNet, FCOS, SSD, …	train: `model(images, targets)` and the sum of the loss dict; eval: `model(images)`; a list of images has no batch dimension
`"huggingface"`	`transformers` models	train: `model(**inputs, labels=targets)` and `outputs.loss`
`"lightning"`	`LightningModules`	train: `training_step((inputs, targets), 0)` and the loss it returns; optimizer from `configure_optimizers()`

- select by name: `check_model(model, x, y, plugin="lightning")`
- none of them imports its framework: torchtest2 still depends only on PyTorch
- explicit arguments (`optimizer=`, `loss_fn=`, …) win over the plugin

Using a ready-made plugin: PyTorch Lightning

```python
import lightning as L

class LitClassifier(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(32, 64), nn.ReLU(), nn.Linear(64, 10))
        self.aux_head = nn.Linear(64, 10)                # added later...

    def forward(self, x):
        return self.net(x)
    def training_step(self, batch, batch_idx):
        x, y = batch
        loss = F.cross_entropy(self(x), y)
        self.log("train_loss", loss)                     # harmless outside a Trainer
        return loss
    def configure_optimizers(self):
        return torch.optim.Adam(self.net.parameters(), lr=1e-3)   # ...but not optimised

report = check_model(LitClassifier(), x, y, plugin="lightning")   # report on the next slide
```

The Lightning plugin: report, details and limits

```
FAIL  optimizer_coverage  2 trainable parameter(s) are not in the optimizer: aux_head.weight, aux_head.bias.
                          Was the optimizer created before these layers were added/replaced?
```

- it calls your `training_step` and uses the first optimizer of `configure_optimizers()`, as the trainer would
- a different batch structure? Tell the plugin how to build it:

```python
from torchtest2.plugins import LightningPlugin

plugin = LightningPlugin(make_batch=lambda inputs, targets: {"x": inputs, "y": targets})
check_model(module, x, y, plugin=plugin)
```

- it supports common `LightningModule` patterns; it does not reproduce the Trainer: no strategies or distributed execution, precision plugins, manual optimisation, several optimizers, schedulers, gradient accumulation or callbacks

A model that does not fit the default

```python
class MultiTask(nn.Module):
    def __init__(self):
        super().__init__()
        self.body, self.cls, self.aux = nn.Linear(32, 64), nn.Linear(64, 10), nn.Linear(64, 1)
    def forward(self, x, y=None):
        h = F.relu(self.body(x))
        if y is None:                       # inference:  model(x)    -> logits
            return self.cls(h)
        return {"cls": F.cross_entropy(self.cls(h), y),           # training: model(x, y)
                "aux": self.aux(h).pow(2).mean()}                 #   -> a dict of LOSSES
```

By default, torchtest2 trains through `loss_fn(model(x), y)`—the inference branch, where the `aux` head never runs:

```
FAIL  gradient_flow   2/6 trainable parameter(s) get no gradient from the loss_fn ...: aux.weight, aux.bias
FAIL  params_change   2/6 trainable parameter(s) did not change after an optimizer step: aux.weight, aux.bias
```

A false alarm: in real training `aux` *is* trained. torchtest2 has to call the model the way you train it.

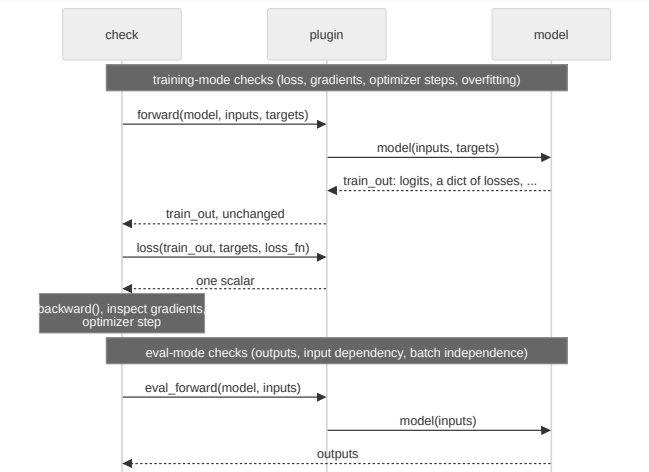
How torchtest2 uses a plugin

```
# every training-mode check (loss, gradients, optimizer steps, overfitting):
train_out = plugin.forward(model, inputs, targets)   # whatever your model returns in training
loss      = plugin.loss(train_out, targets, loss_fn)  # turn THAT into one scalar -> backward()

# every eval-mode check (outputs, input dependency, batch independence):
outputs   = plugin.eval_forward(model, inputs)
```

- `plugin.forward` is not your model's `forward`: it receives the model and decides how to call it
- the first argument of `plugin.loss` is exactly what `plugin.forward` returned—logits for most models, but for `MultiTask` a dict of losses
- `loss_fn` is the loss function given by the user, if any; a plugin may ignore it
- every method has a default: `forward` and `eval_forward` call `model(inputs)`, `loss` applies `loss_fn` (or uses `outputs.loss`)

The plugin calls, step by step



Writing your own plugin

Subclass `torchtest2.Plugin` and override only what differs from the default:

```
from torchtest2 import Plugin

class MultiTaskPlugin(Plugin):
    name = "multitask"
    def forward(self, model, inputs, targets):    # training: call the model WITH the targets
        return model(inputs, targets)              # -> {"cls": ..., "aux": ...}
    def loss(self, losses, targets, loss_fn):      # losses: what forward() returned
        return losses["cls"] + losses["aux"]       # -> one scalar
    # optional: eval_forward (default model(inputs) is right here), configure_optimizer, batch_dim
report = check_model(MultiTask(), x, y, plugin=MultiTaskPlugin())   # all checks pass
```

- the same as `forward_fn=lambda m, x, y: m(x, y)` plus `loss_fn=lambda losses, y: ...` in every test—written once, for the whole model family

Sharing a plugin with your lab

Register it under the `torchtest2.plugins` entry-point group of your package:

```
# pyproject.toml of my_lab_tools
[project.entry-points."torchtest2.plugins"]
multitask = "my_lab_tools.testing:MultiTaskPlugin"
```

After `pip install my_lab_tools`, everyone can write:

```
check_model(model, x, y, plugin="multitask")
torchtest2.plugin_discovery()     # lists the built-in and installed plugins
```

- names are resolved among the built-in plugins first, then only the requested entry point is imported—a broken unrelated plugin cannot break your tests
- plugins also work with `ModelTester` and `TestSuite` (their `split_batch` turns a `batch` into inputs and targets)

Wrap-up

What it does not do

- it tests the model, its loss and its optimizer—not your data pipeline, training loop, metrics or distributed setup—and most real silent bugs reported online live there, outside the model.
- `batch_independence` finds leaks *between* samples, not data scrambled *within* a sample
- vanishing gradients are not flagged yet (only updates lost to rounding)
- a `PASS` is evidence, not a proof: the checks see one batch, before training

Think of it as a linter for model code: cheap, fast, and it catches the embarrassing bugs before they cost you a week of GPU time.

Take-aways and exercises

1. add `assert_model_ok` to the tests of your current project—one test per architecture, with the loss and optimizer you really use
2. break your model on purpose (`.detach()` a layer, swap `batch_first`, drop the attention mask) and read what the report says
3. if you use Lightning or Hugging Face, run the built-in plugin on your module
4. if your lab has its own training convention, write a plugin and share it as a package

Documentation, examples and the bug gallery: the README of the repository.